

Waruka CLI Reference (v1.0.0)

Complete reference for every `python -m waruka <command>` invocation, all parameters, their purpose and effects, and pointers to existing diagnostic visualisations that illustrate the parameters.

Note (v0.13+): `python -m waruka gui` is the recommended entry point for processing matches end-to-end. From v0.16 the GUI is a four-tab shell: Track + Concat + Post-process + Queue. The CLI commands below remain fully supported and are what the GUI drives internally. See the GUI section at the bottom of this document for a quick reference.

Pipeline overview

```
input_video.mp4
■
▼ (interactive)
[1] calibrate ■■■→ project.json (camera intrinsics, dewarp)
■
▼ (interactive)
[2] markfield ■■■→ project.json (homography, field marks)
■
▼ (~3 min)
[3] track ■■■■■■■→ tracks.json (raw per-frame detections, tracked)
■
▼ (~10 sec)
[4] classify ■■■■→ players.json (framing pool: stable-active player tracks)
    ■■■→ players_labeled.json (every track w/ per-frame label)
■
▼ (~10 sec)
[5] campath ■■■■■→ campath.json (smoothed yaw/pitch/hfov per frame)
■
▼ (~90 sec / 120s clip)
[6] render ■■■■■→ broadcast.mp4 (final virtual broadcast view)
```

Or run all 4 in one parallel pipeline (added v0.12, for long matches where the small first-chunk residual is acceptable):

```
[7] pipeline ■■■■■→ broadcast.mp4 (track + classify + campath + render
                                   in parallel worker threads with
                                   chunked input; ~33-50% wall-time
                                   saving vs sequential)
```

Auxiliary commands: `tiles`, `detectpano`, `trackpreview`, `monitor`, `preview`.

Optional post-render step (added v0.14):

```
[8] interpolate ■→ smooth.mp4 (frame-interpolated higher-fps copy of
                               broadcast.mp4 via RIFE 4.25 or FILM)
```

See the interpolate section below.

What's new in v0.16

- **Interpolation perf** -- closes #43. NVDEC source decode +

GPU-resident cached `a_t` across pairs + three-stage thread pipeline + batched dts. Loop wall 19.9 s -> 10.9 s on a 5-s 1440p bench; 100-min match estimate dropped from 6.7 h to 3.6 h. Four new flags expose the knobs:

- `--no-nvdec` -- force cv2 H264 decode instead of NVDEC. Loses

the ~40% wall-time saving; pick this if you want the v0.14 cv2 YUV->BGR matrix for exact colour parity with that era's output.

- `--no-pipeline` -- run the decoder / model / encoder

synchronously instead of in three threads. Debugging / A-B.

- `--no-batch-dts` -- one model call per timestep instead of

batching all dts of a pair into a single forward pass. Debugging / matching v0.15.2 numerics exactly.

- `--cq` -- NVENC constant-quality target (0-51, lower=better).

See #42 below.

- **#42 'RIFE shimmer' fixed -- it was NVENC bitrate starvation.**

The 2-second blur at the start of an interpolated output was the encoder's VBR rate-controller ramping up under a low default bitrate, not a RIFE flow-estimator issue. Fixed by switching `waruka interpolate` to constant-quality VBR (`-rc vbr -cq 23`); CLI flag `--cq` exposed. Sharpness on the user's perftest clip went from 38% below source to within 7%. Note: file sizes are ~5-10x larger at the new default; drop `-cq` to 26-28 to halve the file size with marginal quality loss. CQ 30 is the model-floor point.

- ****waruka upscale** -- new standalone 2x super-resolution

command.** Real-ESRGAN x2plus across every input frame. ~10-15 s/frame at 1440p on a 2080 Ti so realistically for short clips, not whole matches. Output codec switches to HEVC NVENC when 2x dims would exceed H.264's 4096-px limit.

- **Queue tab (#35 closed)** in the GUI. Set up several games

end-to-end (concat list, calibrate + markfield, tracking parameters, optional interpolation), then start the queue before sleeping. Persistent across crashes/restarts (~/.waruka/queue.json); pause / resume at stage boundaries; retry from failed stage; per-job archived logs. Intermediates + project + log live under `<broadcast_dir>/waruka_tracking/<basename>/` so the output directory stays clean.

- **Post-process tab** in the GUI. Run `waruka interpolate`

and/or `waruka upscale` against any existing video. When both are checked, the source is upscaled first (smaller per-frame cost) then interpolated.

What's new in v0.15

- `waruka polish` -> `waruka interpolate`. Renamed for clarity --

the subcommand only ever did frame interpolation, never grew other passes. CLI flag, GUI label, internal stage name, and progress command all updated. No functional change.

- **SR is now constant** (no per-frame bypass). `sr_min_upscale`

default dropped from 1.5 to 0.0 in `waruka render --sr` and the `GpuRenderer` constructor; the threshold can still be set above 1.0 by power users who want the old "skip SR at wide framings" behaviour. The previous default caused a visible "pop" when the bypass triggered.

- **First-pair shimmer fix attempted** (and rolled back conceptually).

A cuDNN-benchmark warmup pass was added to `waruka interpolate` before the real loop -- it does no harm but did NOT fix the ~2-second shimmer at the start of an interpolated output, ruling out cuDNN algorithm selection as the cause. The warmup code stays because it's cheap insurance; root cause is on the backlog as #42.

- **Render-time perf budget on a 100-min match** (RTX 2080 Ti):
- Plain render: 3.4 h (same-day)
- Render + interpolate to 60 fps: 21.7 h (single overnight)
- Render with `--sr`: 10.5 h (single overnight)
- Render with `--sr` + interpolate: 28.1 h (weekend)

What's new in v0.14

- `waruka interpolate` post-render frame interpolation (#18; was

`waruka polish` in v0.14). RIFE 4.25 default backend (~250 ms/pair end-to-end at 1440p, ~16 h for a 100-min match at 3x). FILM-Style available as an opt-in alternative via `--backend film` (slightly cleaner on huge motion but ~4x slower; see warning in the interpolate section).

- `waruka render --sr` source-crop super-resolution (#41) via

Real-ESRGAN x2plus. Per frame, the `GpuRenderer` crops the source pano to the grid bbox, upscales x2, then samples from the upscaled crop. Auto-bypassed per-frame when the source crop is already large enough relative to output (default threshold 1.5x upscale). Visible sharpness improvement at tight action zooms; adds ~150-700 ms per frame at typical campath hfovs. See the render section.

- **GPU** `pad_source_for_blur` (render): the per-frame blur-padding

step that prefixes the `GpuRenderer` in `blur` edge-fill mode now runs on the GPU via torch when CUDA is available. ~5.8x faster (50 ms → 8 ms per call at 1728×4608); CPU fallback preserved bit-for-bit if torch+CUDA aren't present.

- **Drag-drop video onto the Track tab** in the GUI: the

`Source video` group box accepts a single dragged video file anywhere within it; the load path is identical to clicking "Open video..." so artefact-dir detection / `project.json` handling flow unchanged.

What's new in v0.13

- **Full GUI shipped** (`python -m waruka gui`) as the recommended

entry point — PySide6 tabbed shell with **Track tab** (drives `calibrate` → `markfield` → `params` → `process`; auto-skips `calibrate` / `markfield` when `project.json` already has them) and **Concat tab** (multi-clip concatenation + scrubber-based trim, `ffmpeg -c copy concat`, auto-handover to Track tab on Save).

- **Calibrate UI overhaul** — resizable for small screens, added a

loupe (Z toggles, 5× INTER_NEAREST) matching `markfield`'s, render cache so cursor-only motion is responsive.

- **Arrow-key cursor nudge + virtual cursor** in both `calibrate` and

`markfield`. Mouse moves update the virtual cursor; arrow keys nudge by 1 source pixel; click + Enter/Space commit at the virtual cursor.

- **Magnifier mode (M key)** in both `calibrate` and `markfield` —

Windows-only (silent no-op elsewhere): topmost click-through circular borderless loupe centred on the OS cursor, with the default cursor hidden so the drawn crosshair is the sole pointer.

- **YOLO weights resolver** — every `YOLO(model_name)` call goes

through `_resolve_weights_path()`, which returns the absolute path of `yolo11n.pt` in the project root. Avoids fresh CDN downloads when subprocesses use `cwd=<artefact_dir>`.

What's new in v0.12

- **Adaptive Panini d** (campath): per-frame `d` solved from

calibration + per-frame hfov/pitch so projection just barely fits pano vfov budget. Eliminates black "moons" at wide framings. Default ON; cap 1.5.

- **Pano edge fill** (render): three modes (`zeros`/`border`/`blur`)

with progressive blur + fade band hiding the seam at the pano vfov edge. Auto-sized `pad_deg` per calibration. Default `blur`.

- **Batched YOLO detect** (track): ~1.9x predict speedup via single

GPU forward pass over all tiles. Default ON.

- `waruka pipeline`: NEW cross-stage chunked pipeline command.

Cross-chunk tracker state continuity makes boundaries invisible on long matches; ~33-50% wall-time saving vs sequential.

- **Function-default sync:** in-process callers (pipeline, tests,

Python API) now get production behaviour without having to pass every CLI flag explicitly.

Existing diagnostic visualisations in the working dir (0:\Waruka\Claude_viz_*.png / *.py) illustrate many of the parameters described below. They are referenced where applicable.

[1] waruka calibrate

Interactive plumb-line dewarp calibration. Tunes the pano dewarp parameters (`k1`, `hfov_deg`, `vfov_deg`, `pitch0_deg`, `roll0_deg`) against the actual camera by showing a live preview window with sliders and persisted state.

```
python -m waruka calibrate <video> [--project project.json] [--time 2.0]
```

Arg	Default	Purpose
video	required	Source video file. Reolink Duo 2 panorama (4608x1728).
--project	project.json	Output project file (created or updated).
--time	2.0	Seconds into the video to read the calibration frame.

Locked defaults for Reolink Duo 2: `k1=0.071`, `hfov=190`, `vfov=80`, `pitch0=roll0=0`. The fit is human-in-the-loop ("looks like a TV broadcast" by eye, optional auto-fit against the floodlight pylon).

Interactive keybindings (UI overhaul in v0.8, loupe + cursor controls added in v0.13):

Key	Action
, .	Scrub video time by -1 / +1 sec
< >	Scrub by -10 / +10 sec
L	Toggle level reference line
O	Toggle calibration-line overlay (reproject marked lines via current fit)
Z	Toggle loupe (5× zoom window around the virtual cursor; v0.13)
M	Toggle magnifier mode — borderless click-through circular loupe centred on the OS cursor (Windows-only; silent no-op elsewhere; v0.13)
arrow keys	Nudge the virtual cursor by 1 source pixel (v0.13)
Enter / Space	Commit a click at the virtual cursor (v0.13)
0	Re-centre level line vertically
left-drag in preview	pan yaw/pitch
mouse wheel	zoom vfov (clamped 25-110°)
right-drag on level line	move level line vertically
S	Save to project file
Q / Esc	Quit

[2] waruka markfield

Interactive field-mark placement -> least-squares homography. Click 4 corners + sideline points on the raw pano. Outputs a homography that maps pano rays to metric ground (X, Z).

```
python -m waruka markfield <video> [--project project.json]
    [--time 2.0] [--length 100] [--width 37]
    [--cam-height-m FLOAT] [--corner-weights "0.5,0.5,2,2"]
    [--no-auto-balance] [--near-trust 3.0]
```

Arg	Default	Purpose
video	required	Source video file.
--project	project.json	Project file to read/write.
--time	2.0	Sec into video to mark on.
--length	100	Field length incl. endzones (m). WFDF default.
--width	37	Field width (m). WFDF default.
--cam-height-m	None	Known camera mount height (m). Anchors decomposed camera Y in the LSQ. Use only when corner clicks are unreliable (no visible markers at the back corners).
--corner-weights	None (uniform 2.0)	Per-corner LSQ weights C0, C1, C2, C3. Use to downweight noisy back corners e.g. 0.5, 0.5, 2, 2.
--no-auto-balance	(auto-balance on)	Disable MLE per-mark weighting. By default each mark is weighted by inverse local click-error amplification (extreme-longitude marks get less weight).
--near-trust	3.0	Near-sideline trust multiplier. Boosts near-sideline LSQ weights — near sideline is closest to camera and easiest to verify.

Interactive keybindings (v0.8 UI; cursor / magnifier additions in v0.13):

Key	Action
left-click in pano	Place mark (C0/C1/C2/C3 in corners mode, A/B/T for near/far/auto in sideline mode)
A	Switch to NEAR-sideline marking mode
B	Switch to FAR-sideline marking mode
T	AUTO mode (default after 4 corners): each click auto-classified via validator H
, . < >	Frame scrubbing ($\pm 1s/\pm 10s$)
G	Toggle guide polylines (live near/far sideline projection)
H	Toggle fit-box overlay (yellow LSQ outline)
Z	Toggle loupe
M	Toggle magnifier mode (Windows-only; same behaviour as in calibrate; v0.13)

Key	Action
arrow keys	Nudge the virtual cursor by 1 source pixel (v0.13)
Enter / Space	Commit a click at the virtual cursor (v0.13)
click + drag	Move any mark
right-click	Remove nearest mark in current mode
R	Remove all BAD-flagged marks in current mode
F	Print per-point Z table for diagnosis
S	Save to project file
Q / Esc	Quit

Per-click colour feedback: green = residual <1m, yellow = <5m, red = >=5m.

Typical residuals on a well-marked clip: near_rms ~0.1-0.3 m, far_rms ~1-3 m, corner_rms ~1-5 m. See [project-waruka-dewarp- ceiling memory] for why these floors exist (mark quality, not model).

[3] waruka track

YOLO11n detection on rectilinear tile cutouts of the pano, fused into a global ground-space Kalman tracker. Writes tracks.json.

```
python -m waruka track [--project project.json] [--video VIDEO]
  [--stride 3] [--t0 0] [--t1 SECS] [--out tracks.json]
  [--conf 0.50] [--iou 0.5]
  [--fuse-lat-m 0.6] [--fuse-rad-m 2.5]
  [--max-coast-s 0.3] [--min-hits 5]
  [--stationary-pos-spread-m 0.5] [--stationary-min-duration-s 5]
  [--phantom-window-s 2.5] [--phantom-max-spread-m 0.1] [--phantom-max-tiles 8]
  [--down-pad-deg 20] [--tile-h-near 960]
  [--rows 1] [--tile-h-single N]
  [--decoder auto]
  [--batched-predict | --no-batched-predict]
```

Arg	Default	Purpose
--project	project.json	Project file (defines field + dewarp).
--video	from project	Override the video path.
--stride	3	Detection cadence. Detection runs every Nth source frame; tracker interpolates per-frame output. Higher stride = faster but fragmented tracks.
--t0 / --t1	0 / end	Time window in seconds.
--out	tracks.json	Output file.
--conf	0.50	YOLO confidence threshold. Production default — lower (e.g. 0.20) to catch borderline detections at the cost of more false positives.
--iou	0.5	YOLO NMS IoU. Lower suppresses same-player double boxes within a tile.
--fuse-lat-m	0.6	Per-frame fusion lateral (cross-bearing) tolerance, m. Tighter = more split tracks; looser = merging distinct players.

Arg	Default	Purpose
<code>--fuse-rad-m</code>	2.5	Per-frame fusion radial (depth) tolerance, m.
<code>--max-coast-s</code>	0.3	Max time a track may coast past its last real detection before its output dots are suppressed.
<code>--min-hits</code>	5	Real-detection hits required before a track is emitted. Production default; pairs with <code>--conf 0.50</code> to kill single-frame YOLO blips cleanly.
<code>--stationary-pos-spread-m</code>	0.5	Drop tracks whose median position spread is below this over <code>--stationary-min-duration-s</code> . Kills fixed-object false positives (kit, cones, tripods).
<code>--stationary-min-duration-s</code>	5	Duration before stationary-track filter applies.
<code>--phantom-window-s</code>	2.5	Per-frame phantom-segment filter window (sec). Production default; set 0 to disable. At each emit frame, looks at real hits within $\pm$ window; if all are within <code>--phantom-max-tiles</code> AND positionally tight, the dot is suppressed. Catches ID-hijacked phantom segments the whole-track filter misses.
<code>--phantom-max-spread-m</code>	0.1	Phantom filter spread threshold.
<code>--phantom-max-tiles</code>	8	Phantom filter: max unique tiles per hit (1 = single-tile only).
<code>--down-pad-deg</code>	20	How far below the closest field ground point NEAR-row tiles extend (deg).
<code>--tile-h-near</code>	960	NEAR-row tile pixel height.
<code>--rows</code>	1	Number of tile rows. Production default since v0.6 (single tall row, no NEAR/FAR cross-row ambiguity).
<code>--tile-h-single</code>	<code>tile_h_near + 720</code>	Single-row tile height (only with <code>--rows 1</code>).
<code>--decoder</code>	auto	Video decoder. auto (NVDEC if available), nvdec, opencv, cpu.
<code>--batched-predict</code>	on	(v0.12) Single batched YOLO forward pass across all tiles (~1.9x predict speedup).
<code>--no-batched-predict</code>	—	(v0.12) Revert to per-image <code>model.predict(list)</code> for A/B comparison. Detections numerically equivalent up to NMS sort-order noise.

With all production defaults baked in, the recommended command simplifies to:

```
python -m waruka track --project project_input_video_short_N.json \
    --t0 0 --t1 120 --out tracks_N.json
```

(All of `--stride 3`, `--conf 0.50`, `--min-hits 5`, `--phantom-window-s 2.5`, `--phantom-max-spread-m 0.1`, `--phantom-max-tiles 8`, `--rows 1` are now the defaults — no need to pass them.)

[4] waruka classify

On-field vs sideline behavioural classifier. Reads `tracks.json`, writes `players.json` (framing pool: stable-active player-labelled tracks only — consumed by campath) AND `players_labeled.json` (every track every frame with label — consumed by render overlay).

```
python -m waruka classify <tracks> [--project project.json]
      [--out players.json] [--buffer 1.0] [--overlay-times "10,30"]
```

Arg	Default	Purpose
tracks	required	Input tracks file from waruka track.
--project	project.json	Project file (defines field rectangle).
--out	players.json	Output framing pool. players_labeled.json auto-derived.
--buffer	1.0	In-field margin for general use (m).
--overlay-times	None	Comma-separated seconds to dump per-frame classification overlay PNGs to _inspect/.

Classifier architecture (not CLI-exposed but configurable in code)

Three layered gates (`classify_tracks()` params in `waruka/classify.py`):

Per-track lifetime label ("player" / "sideline" / "foreign"):

- `deep_m=5.0` — deep zone is Z in `[5, W-5]` (~middle 27m of width).
- `min_deep_run_s=2.0` — min consecutive deep time to qualify as player.
- `min_deep_run_sideline_s=7.0` — stricter threshold for tracks anchored near a sideline.
- `sideline_band_m=5.0` — defines "sideline-anchored" (`z_med < 5` or `> W-5`).
- `static_min_s=6.0 + min_move_m=0.5` — stationary veto (≥ 6 s with < 0.5 m `pos_spread` → sideline).
- `far_static_spread_m=2.0` — looser stationary threshold for far-sideline tracks (wobble + small angular size).

Per-frame Schmitt active gate (player-labelled tracks only):

- `active_margin_near=-0.5`, `active_margin_far=-2.0`, `active_margin_ends=+2.0` — activation boundaries (must be inside these to (re-)activate).
- `active_hyst_band_m=1.5` — hysteresis band; deactivation boundary is activation + band.
- `active_off_hold_near_s=0.3`, `active_off_hold_far_s=0.3`, `active_off_hold_ends_s=10.0` — per-direction time required clear-off before deactivation. Endzone walk-out gets 10s grace.

Committed-grace gate:

- `committed_grace_enabled=True` — master toggle.
- `committed_off_hold_s=7.0` — extended sideline hold for committed tracks.
- `committed_min_deep_s=3.0` — must spend this much time in deep zone within the window.
- `committed_recent_window_s=5.0` — rolling window length. Decays after ID-switch.

Probation gate (suppresses brief sideline-walk-in flashes):

- `probation_s=3.0` — must stay continuously active this long.
- `isolated_dist_m=8.0` — fresh activation isolated by > this from cluster goes into probation.
- `well_inside_margin_near=-1.0`, `well_inside_margin_far=-3.0`, `well_inside_margin_ends=+2.0` — promotion zone (must spend `probation_s` here, cumulative).

Output labels in `players_labeled.json`:

- `"player"` → green dot in render (in framing pool).
- `"probation"` → yellow dot (excluded from framing pool).
- `"sideline"` / `"foreign"` → red dot (excluded).

See `0:\Waruka\Claude\viz_well_inside_clip1_t7.png` and `_viz_committed_grace_clip1_v13.png` for visual examples of the classifier boundaries.

[5] waruka campath

Plans a smoothed camera path from the framing pool. Consumes `players.json` (NOT raw tracks — see [feedback-layering-trust- classifier memory]). Writes `campath.json` with per-frame yaw/pitch/hfov + projection mode + Panini `d` parameter.

```
python -m waruka campath <players> [--project project.json]
    [--margin DEG] [--hfov-min DEG] [--view-mode default|wide]
    [--panini-preset rectilinear|panini] [--panini-d FLOAT]
    [--panini-d-adaptive | --no-panini-d-adaptive]
    [--panini-d-cap FLOAT] [--panini-d-safety DEG]
    [--panini-d-black-tolerance DEG] [--panini-d-min-threshold FLOAT]
    [--out campath.json]
```

Arg	Default	Purpose
<code>players</code>	required	Framing pool from <code>waruka classify</code> .
<code>--project</code>	<code>project.json</code>	Project file (defines pano, field).
<code>--margin</code>	preset's choice	Angular margin per side in degrees. Overrides preset's <code>margin_deg</code> .
<code>--hfov-min</code>	preset's choice	Floor on output hfov in degrees. Overrides preset's <code>hfov_min</code> .
<code>--view-mode</code>	default	Named preset. <code>default</code> = { <code>hfov_min</code> =26, <code>margin_deg</code> =8} (tight, natural). <code>wide</code> = { <code>hfov_min</code> =26, <code>margin_deg</code> =15} (more breathing room).
<code>--panini-preset</code>	rectilinear	Static Panini <code>d</code> preset: <code>rectilinear</code> (<code>d</code> =0.0, straightest lines) or <code>panini</code> (<code>d</code> =1.0, classic stereographic). Used as the base value when adaptive is OFF.
<code>--panini-d</code>	None	Explicit Panini <code>d</code> . Overrides <code>--panini-preset</code> .
<code>--panini-d-adaptive</code>	on	(v0.12) Per-frame <code>d</code> solved from calibration + per-frame hfov/pitch to just fit the pano vfov budget. Writes per-frame <code>d</code> into the campath JSON. Default ON.
<code>--no-panini-d-adaptive</code>	—	(v0.12) Use a static <code>d</code> for the whole clip (the <code>--panini-preset</code> or <code>--panini-d</code> value).

Arg	Default	Purpose
<code>--panini-d-cap</code>	1.5	(v0.12) Upper bound on the adaptive d. $d > 1.5$ looks visibly cylindrical; below that it's hard to tell from rectilinear at typical play distances.
<code>--panini-d-safety</code>	2.0	(v0.12) Degrees of pano vfov reserved as buffer (avoid sampling artefacts at the very edge).
<code>--panini-d-black-tolerance</code>	0.0	(v0.12) Degrees of ray-pitch overflow tolerated before d engages. 0 = strict no-black; raising it keeps $d=0$ over a wider HFOV range at the cost of a black sliver. Default 0 because a smooth d-ramp through small values reads more naturally than a sharper engage.
<code>--panini-d-min-threshold</code>	0.0	(v0.12) Snap-to-zero floor on the smoothed d. Default off because the smoother's deadzone can oscillate around the threshold (worse than the asymptotic tail).
<code>--out</code>	<code>campath.json</code>	Output path.

Adaptive Panini d — what it does

The Panini-General projection has a tunable `d` parameter that trades line-straightness for edge stretch. At wide framings (large hfov), rectilinear ($d=0$) demands ray pitches beyond the pano's vfov, producing visible black "moons" at top/bottom of the output.

Adaptive d solves a closed-form per frame: given the calibration's `vfov_pano` and `pitch0`, plus the per-frame virtual hfov and pitch, find the minimum d such that the top-centre and bottom-centre rays fit within the pano's vfov budget (minus 2° safety).

- At narrow HFOV ($< \sim 80^\circ$): $d = 0$ (pure rectilinear).
- At intermediate HFOV: d ramps smoothly toward the cap.
- At extreme HFOV past the Panini family asymptote: d caps at 1.5,

and any residual is filled by the renderer's edge fill (see `waruka render`).

The result is invisible-quality framing at narrow zoom and natural- looking projection at wide zoom, automatically per frame.

Function-only parameters (not CLI-exposed, can be set via Python)

`plan_campath()` in `waruka/campath.py` accepts many more knobs:

Param	Default	Purpose
<code>lookahead_s</code>	2.5	Lookahead window for anticipation.
<code>smooth_t</code>	0.7	Critically-damped follow time constant (s).
<code>hfov_max</code>	180.0	Hard ceiling on hfov.
<code>dropout_hfov_frac</code>	0.5	Threshold: natural hfov dropping to <this fraction of last valid hfov triggers dropout detection.
<code>max_dropout_hold_s</code>	4.0	Max time to hold last-valid framing during a dropout.

Param	Default	Purpose
yaw_deadzone_deg	2.0	Smoothing deadzone for yaw (°).
pitch_deadzone_deg	1.5	Smoothing deadzone for pitch (°).
hfov_deadzone_deg	3.0	Smoothing deadzone for hfov (°).
soft_deadzone	True	Quadratic spring-force taper inside deadzone (smooth pans instead of stair-step).
lookahead_aggregator	"median"	"median" (commit only if >50% of window shifts) or "mean".
projection_mode	"panini"	"panini" (default), "cylindrical" (legacy), "rectilinear" (pure pinhole).
panini_preset	"rectilinear"	"rectilinear" (d=0.0) or "panini" (d=1.0). Used when adaptive is off.
panini_d	None	Explicit Panini d. Overrides preset.
initial_smoother_state	None	(v0.12) Dict of {yaw_pos, yaw_vel, pitch_pos, pitch_vel, hfov_pos, hfov_vel, d_pos, d_vel} to seed the smoother. Used by waruka pipeline for cross-chunk smoother continuity. Also writes smoother_final_state to the campath JSON.
d_smooth_t	0.4	(v0.12) Smoothing time constant for the per-frame d signal.
d_deadzone	0.05	(v0.12) Deadzone width on the d smoother.

See diagnostic charts:

- `_viz_campath_v08_vs_v09.png` (mean lookahead vs classifier-pool)
- `_viz_soft_deadzone.png` (hard vs soft deadzone, slow-pan response)
- `_viz_lookahead_explainer.png` (median vs mean aggregation)
- `_viz_clip4_drop_swing.png` (dropout detection in action)
- `_viz_17_projection_blend.png` (cyl blend = 0, 0.3, 0.6, 1.0 comparison)
- `_viz_sideline_curve.png` (cylindrical hybrid vs pure rectilinear)

[6] waruka render

Renders the final virtual broadcast camera view. Two modes:

1. **Broadcast (default)**: 2560×1440 cropped virtual camera using the campath's per-frame yaw/pitch/hfov + projection.
2. **Debug-pano** (`--debug-pano`): full panorama (downscaled) with yellow crop-box overlay, cyan field perimeter, and (with `--overlay-tracks`) coloured dots from a labelled tracks file.

```
python -m waruka render <campath> [--project project.json] [--video V]
    [--out OUT] [--overlay-tracks LABELLED]
    [--t0 SECS] [--t1 SECS]
```

```

[--debug-pano] [--debug-pano-width 2560] [--plain-dots]
[--show-raw-yolo] [--det-conf 0.20] [--det-iou 0.5]
[--rows 1] [--down-pad-deg 20] [--tile-h-near 960] [--tile-h-single N]
[--pano-edge-fill zeros|border|blur]
[--pano-edge-fill-blur-deg DEG] [--pano-edge-fill-blur-sigma PX]

```

Arg	Default	Purpose
campath	required	Campath file from waruka campath.
--project	project.json	Project file.
--video	from project	Override video path.
--out	broadcast.mp4	Output MP4.
--overlay-tracks	None	Labelled tracks file (players_labeled.json) for dot overlay. Required for coloured dots.
--t0 / --t1	window from campath	Trim window.
--debug-pano	off	Render the full pano with crop box overlay instead of the virtual camera.
--debug-pano-width	2560	Downscale width for debug-pano output.
--plain-dots	off	Force single-colour cyan dots (no green/red/yellow). Useful for raw-tracks-without-classifier debug.
--show-raw-yolo	off	(debug-pano only) Overlay raw per-tile YOLO boxes.
--det-conf	0.20	(with --show-raw-yolo) YOLO conf for raw boxes.
--det-iou	0.5	(with --show-raw-yolo) NMS IoU.
--rows	1	(with --show-raw-yolo) Tile rows. Must match track's --rows.
--down-pad-deg	20	(with --show-raw-yolo) Must match track's value.
--tile-h-near	960	(with --show-raw-yolo) Must match.
--tile-h-single	tile_h_near + 720	(with --show-raw-yolo, --rows 1) Must match.
--pano-edge-fill	from project (blur)	(v0.12) Override edge-fill mode: zeros (legacy black), border (clamp to edge row), blur (progressive blur + fade band — recommended).
--pano-edge-fill-blur-deg	from project (auto)	(v0.12) Vfov extension (deg) per side for blur mode. None = auto-compute from calibration + campath (compute_required_pad_deg).
--pano-edge-fill-blur-sigma	from project (40)	(v0.12) Horizontal Gaussian blur sigma (px) at the outermost padded row.

Source-crop super-resolution (--sr, v0.14, #41)

Per frame, the renderer crops the source pano to the bbox of the sampling grid and runs Real-ESRGAN x2plus on that crop before the final grid_sample to output. Net effect: tight action zooms where the source crop is smaller than the output get a real sharpness improvement instead of the bicubic softness of a 2-4× upscale.

```
python -m waruka render campath.json --project P.json \
  --out broadcast.mp4 --sr [--sr-min-upscale 1.5]
```

- `--sr` -- enable source-crop SR. Off by default.
- `--sr-min-upscale 1.5` -- only run SR when the natural upscale

ratio (max of `out_w / crop_w`, `out_h / crop_h`) is at least this. Default 1.5 means SR runs when the source crop is at least 1.5x smaller than the output in either axis. Lower = SR runs more often (more compute); higher = SR runs less.

When SR helps: tight zooms ($\text{hfov} \leq \sim 70^\circ$, typical action plays). **When SR is bypassed:** wide framings ($\text{hfov} \geq \sim 75^\circ$) -- the source already has more pixels than the output, so adding SR would only cost compute without quality gain. Each frame's decision is taken from the actual grid bbox, not the hfov, so the bypass is exact rather than a heuristic.

Cost: linear in source crop pixels.

hfov	source crop	SR time
20° (tight)	486×270	~80 ms
40°	970×540	~300 ms
60°	1450×810	~680 ms
80°	bypassed	(no SR)

Total render budget rises ~5-10× when SR is active. For a 100-min match at 1440p that's roughly 13 h of render time -- single-overnight, not real-time. Pairs naturally with `waruka interpolate` after the render finishes (~16 h additional for RIFE 3x to 60 fps).

Not currently wired into `waruka pipeline`. Pipeline mode runs the non-SR path; use the sequential `track → classify → campath → render chain` (or the GUI's sequential mode) when SR is wanted.

Model: `RealESRGAN_x2plus.pth` (~66 MB) at `third_party/realrgan/weights/`. BSD-3 license. Architecture vendored at `third_party/realrgan/realrgan/archs/rrdbnet_arch.py` (standalone RRDBNet, no basicsr dependency).

Edge-fill modes (v0.12)

At extreme HFOVs the projection asks for rays beyond the pano's vfov coverage. Even with adaptive d (see `waruka campath`), the cap (`panini_d_cap=1.5`) means some residual demand remains. The renderer's edge-fill controls how this residual is filled.

- `zeros` (legacy): black pixels. Visible "moons" at top/bottom

of wide framings.

- `border: grid_sample padding_mode="border"` — clamps to the

nearest edge row. Visually a stretched-up edge row. Cheap, but introduces vertical streaks from any cloud edges or treeline detail in the boundary row.

- `blur` (default): pre-pad the source frame on CPU. The padded

region is a progressively-blurred extension of the boundary row (sigma ramps from `boundary_sigma=8` near the seam to `sigma_max=40` at the top of the pad). A small fade band inside the original (`fade_deg=2`) hides the seam by blending the topmost original rows toward the boundary-blurred copy. Pre-pad size auto-derived per render: `auto pano_edge_fill_blur_deg = 12.6` (`vfov_pano=78.0`, `pitch0=-15.7`) printed at start.

Border-clamp also serves as a safety net under `blur` mode — if the auto `pad_deg` underestimates anywhere, `grid_sample padding_mode="border"` clamps to the (already-blurred) outermost padded row.

Dot colours in `--overlay-tracks mode`

(From v0.9 classifier output via `waruka classify`):

- **Green** = stable active (in framing pool, drives camera).
- **Yellow** = probation (Schmitt says active but isolated/unconfirmed).
- **Red** = sideline / foreign / inactive-player.

Pass `--plain-dots` to force single-colour cyan (for raw `tracks.json` overlay without labels).

Projection modes (read from campath JSON)

The campath JSON's "projection" field controls render math. The `panini_d` field (when present) sets the Panini d parameter.

- "panini" with `panini_d=0.0` (default) — pure rectilinear via the

Panini formula. Straightest lines, mild edge stretch at wide FOV.

- "panini" with `panini_d=1.0` — classic stereographic Panini.

Less line-straight but less edge stretch.

- "cylindrical" — legacy hybrid (rect-x + cyl-y). Uses

`cfg.projection_blend` as the rect/cyl mix.

- "rectilinear" — pure pinhole; FOV interpreted as VFOV (legacy).

See `_viz_17_projection_blend.png` and `_viz_sideline_curve.png` for visual examples.

[7] waruka pipeline (v0.12)

Cross-stage chunked pipeline: track + classify + campath + render in one CLI call, running as concurrent worker threads with chunked input. Each chunk feeds the next stage's queue as soon as it's ready. Final concat via the bundled `imageio_ffmpeg ffmpeg`.

```
python -m waruka pipeline [--project project.json] [--video V]
    [--out broadcast.mp4]
    [--t0 0] [--t1 SECS]
    [--chunk 30] [--pre-overlap 0] [--post-overlap 0]
    [--work-dir _pipeline_chunks] [--keep-chunks]
    [--no-cross-chunk-state]
```

Arg	Default	Purpose
<code>--project</code>	<code>project.json</code>	Project file.
<code>--video</code>	from project	Override the video path.
<code>--out</code>	<code>broadcast.mp4</code>	Final concatenated MP4.
<code>--t0 / --t1</code>	0 / end	Time window in seconds.
<code>--chunk</code>	30.0	Chunk size in seconds. Larger = fewer chunks (less classifier-divergence at boundaries on edge cases), but lower pipeline depth (less parallel benefit). Default 30 works well on long matches; for short clips prefer sequential.
<code>--pre-overlap</code>	0.0	Per-chunk pre-overlap (sec). Replaced by cross-chunk-state continuity (defaults to 0); kept as opt-in for A/B comparison.
<code>--post-overlap</code>	0.0	Per-chunk post-overlap (sec). Replaced by cross-chunk-state continuity (defaults to 0); kept as opt-in for A/B comparison.

Arg	Default	Purpose
<code>--work-dir</code>	<code>_pipeline_chunks</code>	Directory for per-chunk intermediates (tracks_NNNN.json, players_NNNN.json, campath_NNNN.json, broadcast_NNNN.mp4, tracker_state_NNNN.json, backemit_NNNN.json, tracks_cumulative.json).
<code>--keep-chunks</code>	off	Keep per-chunk intermediates at end. Default deletes them after successful concat.
<code>--no-cross-chunk-state</code>	(on)	Disable cross-chunk tracker + classifier state continuity. Legacy / A-B comparison. Causes large boundary jumps -- only use to verify the fix is in effect.

Cross-chunk continuity (v0.12)

When `--cross-chunk-state` is on (default), the pipeline threads several pieces of state across chunk boundaries so the output is near-equivalent to a single-pass run:

1. **Tracker state.** Active tracks (Kalman state + hits + IDs)

serialise to `tracker_state_NNNN.json` and seed the next chunk's tracker. Track IDs persist across boundaries.

2. **Smoother state handoff.** `plan_campath` reads

`initial_smoother_state` from the previous chunk's campath at the render boundary frame and writes `smoother_final_state` to its own. Camera pan/zoom is continuous across boundaries.

3. **Cumulative tracks file.** Each chunk's emit is appended to

`tracks_cumulative.json`. The classifier always reads this cumulative file -- it sees full per-track lifetimes regardless of chunk boundaries.

4. **Lookahead-by-1 classify.** Chunk N's classify is buffered

until chunk N+1's tracks have been added to cumulative -- so chunk N has the same future evidence chunk N+1 will have at the boundary.

5. **Back-emit merge.** Chunk N+1's tracker also re-emits chunk

N's range using N+1's hits (better interpolation across the boundary). Pipeline merges these into `tracks_cumulative.json`, replacing alive-track entries.

Result on a 4-chunk 120s clip 3 test (vs fresh single-pass):

chunk	yaw RMS	hfov RMS	max yaw
0 (0-30s)	4.6°	8.1°	10°
1 (30-60s)	0.02°	0.50°	0.18°
2 (60-90s)	0.00°	0.00°	0.00°
3 (90-120s)	0.00°	0.05°	0.04°

Boundary jumps all sub-0.1°. Pipeline wall ~33% saving vs sequential on this 4-chunk run; scales toward ~50% on long matches (more chunks = deeper pipeline = more overlap of track + render across the chunk sequence).

Known limitation: chunk-0 residual

The FIRST chunk (chunk 0) has ~5° yaw RMS divergence from single-pass for the first ~20s of its render. Root cause is YOLO/FP16 inference non-determinism: two close YOLO boxes (~1m apart in ground space) sometimes fuse into one track and sometimes stay as two. The chunked tracker inherits this non-determinism.

For short clips (< 2 min) where chunk 0 dominates a large fraction of the output, use the classic sequential workflow (track → classify → campath → render). `waruka pipeline` is for long matches where the first-20s region is a negligible fraction of the total.

A future fix ("detect once, track chunked": store YOLO detections in an intermediate file, then chunk only the tracker+classify+ render off that single file) is sketched in the project memory -- estimated ~200 LOC.

Pipeline output

<code>broadcast.mp4</code>	<code># final concatenated output</code>
<code>_pipeline_chunks/</code>	
<code> tracks_NNNN.json</code>	<code># per-chunk tracker output</code>
<code> tracker_state_NNNN.json</code>	<code># active tracks + IDs at chunk end</code>
<code> backemit_NNNN.json</code>	<code># chunk N+1's re-emit for chunk N range</code>
<code> tracks_cumulative.json</code>	<code># merged cumulative tracks file</code>
<code> players_NNNN.json</code>	<code># per-chunk classifier output</code>
<code> campath_NNNN.json</code>	<code># per-chunk camera path + smoother state</code>
<code> broadcast_NNNN.mp4</code>	<code># per-chunk render</code>

(All cleaned up by default unless `--keep-chunks`.)

Auxiliary commands

`waruka tiles`

Dump detection tiles at one moment, with all YOLO boxes annotated (class, conf, edge-cut, projected X/Z or DROP reason). Used for single-frame instrumentation before any long rerun.

```
python -m waruka tiles [--project project.json] [--video V]
                        [--time 2.0] [--out _tiles] [--conf 0.20] [--iou 0.5]
```

`waruka detectpano`

Single-frame pano with detection boxes + tracked feet dots + IDs. Auto-uses stored boxes from `tracks.json` (~5 s) when present; falls back to live YOLO otherwise (~45 s).

```
python -m waruka detectpano [--project project.json] [--video V]
                             --time T [--tracks tracks.json] [--out _detpano.png]
                             [--conf 0.20] [--iou 0.5]
```

`waruka trackpreview`

Overlay a tracks file on a raw panorama at one time. Quick visual sanity check for tracks before classify/campath.

```
python -m waruka trackpreview <tracks> [--project project.json]
                        [--time 2.0] [--out track_overlay.png]
```

`waruka monitor`

Tkinter GUI showing live progress of an in-flight track/render (reads `_progress.json`). Run alongside `waruka track` or `waruka render`. Shows step, progress bar, ETA, fps, per-tile counts, with a Kill button.

```
python -m waruka monitor [--path _progress.json]
```

`waruka preview`

Headless dewarp preview to a PNG. Useful for sanity-checking the calibrated dewarp without launching the interactive calibrate UI.

```
python -m waruka preview <video> [--project project.json]
                        [--time 2.0] [--yaws "-45,0,45"] [--vfov 75.0]
```

Per-project file (project.json)

Persisted between runs. Contains:

- `source_video`: relative path to the source video.
- `pano`: `{src_w, src_h, hfov_deg, vfov_deg, pitch0_deg, roll0_deg, k1, k2, cx, cy}` — dewarp model.
- `out_w, out_h`: virtual camera output size (default 2560×1440).
- `homography`: 3×3 row-major flattened, ground↔ray mapping.
- `field_marks`: `{corners, near, far}` arrays of pano pixel clicks.
- `field_length_m, field_width_m`: WFDF defaults 100, 37.
- `projection_blend`: cylindrical-mode blend factor.
- `panini_d`: Panini d parameter (default 0.0; used when adaptive is off).
- `panini_d_adaptive`: bool, default `true` (v0.12).
- `panini_d_cap`: float, default 1.5 (v0.12).
- `panini_d_safety_deg`: float, default 2.0 (v0.12).
- `panini_d_black_tolerance_deg`: float, default 0.0 (v0.12).
- `panini_d_min_threshold`: float, default 0.0 (v0.12).
- `pano_edge_fill_mode`: "zeros" | "border" | "blur", default "blur" (v0.12).
- `pano_edge_fill_blur_deg`: float or null (auto), default `null` (v0.12).
- `pano_edge_fill_blur_sigma_px`: float, default 40.0 (v0.12).
- `pano_edge_fill_blur_boundary_sigma_px`: float, default 8.0 (v0.12).
- `pano_edge_fill_blur_fade_deg`: float, default 2.0 (v0.12).
- `cam_height_m, corner_weights`: opt-in refinement.
- `auto_balance_marks, near_trust`: MLE fit knobs.
- `last_scrub_t, show_guides, show_fitbox,`
`level_line_y_frac`, etc. — UI state persistence for calibrate + markfield.

Typical full-clip command chain

```
N=2 # clip number
PROJ=project_input_video_short_${N}.json

# 1. Calibrate (interactive, one-time)
python -m waruka calibrate input_video_short_${N}.mp4 --project ${PROJ}

# 2. Mark field (interactive, one-time)
python -m waruka markfield input_video_short_${N}.mp4 --project ${PROJ}

# 3. Track (production defaults baked in)
python -m waruka track --project ${PROJ} \
    --t0 0 --t1 120 --out tracks_${N}.json

# 4. Classify
```

```
python -m waruka classify tracks_${N}.json \
  --project ${PROJ} --out players_${N}.json

# 5. Campath
python -m waruka campath players_${N}.json \
  --project ${PROJ} --out campath_${N}.json

# 6a. Debug-pano (green/yellow/red dots)
python -m waruka render campath_${N}.json --project ${PROJ} \
  --out broadcast_${N}_debug.mp4 \
  --debug-pano --overlay-tracks players_${N}_labeled.json

# 6b. Camera-view (the broadcast)
python -m waruka render campath_${N}.json --project ${PROJ} \
  --out broadcast_${N}.mp4
```

Total wall time on RTX 2080 Ti (120s clip): ~5 min for steps 3-6.

Or, for long matches: chunked pipeline (v0.12)

For a single match longer than ~5 min where ~5° framing divergence in the first 20s is acceptable:

```
python -m waruka pipeline --project ${PROJ} \
  --t0 0 --t1 6000 --chunk 30 --out broadcast.mp4
```

~33-50% wall-time saving vs the sequential chain on long matches. For short clips (< 2 min), the chunk-0 residual covers too much of the output — stick with the sequential chain above.

Common diagnostic visualisations

Pre-existing scripts in the working dir (run them or just inspect the PNGs):

Script / PNG	Demonstrates
<code>_viz_boundaries_t70.png</code>	Schmitt activation/deactivation boundaries on a frame
<code>_viz_well_inside_clip1_t7.png</code>	Probation well_inside vs activation boundary
<code>_viz_deep_vs_central.png</code>	"Deep zone" vs "central band" terminology
<code>_viz_committed_grace_clip1_v13.png</code>	Per-track timeline strip across classifier versions
<code>_viz_lookahead_explainer.png</code>	Median vs mean lookahead aggregation
<code>_viz_campath_v08_vs_v09.png</code>	Per-clip campath A/B over time
<code>_viz_soft_deadzone.png</code>	Hard vs soft deadzone, slow pan response
<code>_viz_clip4_drop_swing.png</code>	Dropout-detection causing camera swing
<code>_viz_hfov_smoothing_damp.png</code>	Smoothing-damping effect on rare events
<code>_viz_17_projection_blend.png</code>	Cylindrical blend = 0, 0.3, 0.6, 1.0
<code>_viz_sideline_curve.png</code>	Cylindrical hybrid vs pure rectilinear
<code>_viz_campath_filter_bug_t48.png / _fixed_t48.png</code>	Before/after #29 fix

Stitched side-by-side comparison videos in the working dir: `_broadcast_clipN*_sidebyside.mp4`.

[9] waruka interpolate

Frame-interpolate a rendered broadcast video to a higher fps using a deep-learning interpolator. Designed as a post-render step — does not touch GpuRenderer. Added v0.14; renamed from `waruka polish` in v0.15.

```
python -m waruka interpolate broadcast.mp4 --out smooth.mp4 [--fps 60]
    [--backend rife|film] [--model PATH] [--fp32] [--no-tile]
    [--t0 SEC] [--t1 SEC]
    [--no-nvdec] [--no-pipeline] [--no-batch-dts] [--cq 23]
```

Arg	Default	Purpose
<code>input</code>	required	Source video to interpolate (e.g. <code>broadcast.mp4</code> from waruka render).
<code>--out</code>	required	Output mp4 path.
<code>--fps</code>	60.0	Target output fps. Must be an integer $\geq 2\times$ multiple of source fps. With the default 20-fps render this means 40 (2x), 60 (3x), 80 (4x), etc.
<code>--backend</code>	rife	Interpolation model. <code>rife</code> = RIFE 4.25 (recommended). <code>film</code> = FILM-Style (see warning below).
<code>--model</code>	None	Override model location. For <code>rife</code> : directory containing <code>train_log/flownet.pkl</code> . For <code>film</code> : path to TorchScript .pt.
<code>--fp32</code>	off	Use float32 instead of float16. Slower; debugging only.
<code>--no-tile</code>	(auto)	FILM-only: disable tile-stitch. Default auto-tiles when source width ≥ 1920 to dodge the 1440p cuDNN kernel-cliff. RIFE never tiles.
<code>--t0, --t1</code>	None	Process only the time window [<code>t0</code> , <code>t1</code>] seconds.
<code>--no-nvdec</code>	(NVDEC on)	Force cv2 H264 decode instead of NVDEC. Slower (~40% wall-time penalty at 1440p) but matches cv2's YUV->BGR matrix; pick this if you want exact colour parity with v0.14 output.
<code>--no-pipeline</code>	(pipeline on)	Disable the three-stage (decoder / model / encoder) thread pipeline and run the loop synchronously. Debugging / A-B against the v0.15.1 sequential baseline.
<code>--no-batch-dts</code>	(batched)	Run one model call per timestep instead of batching all dts of a pair into a single forward pass. Debugging / matching v0.15.2 numerics.
<code>--cq</code>	23	NVENC constant-quality target (0-51, lower=better). Default 23 keeps interp sharpness close to the source. Drop to 26-28 to halve file size with marginal quality loss; 30 is the model-floor point. Fixes the pre-v0.16 "shimmer" / first-2s blur.

Perf, post-v0.16 (RTX 2080 Ti, 1440p, RIFE 4.25):

Path	Loop wall (5-s bench)	ms/pair	100-min match estimate
cv2 + sync (pre-v0.16)	19.9 s	200	~6.7 h

Path	Loop wall (5-s bench)	ms/pair	100-min match estimate
NVDEC + GPU-resident (v0.16.1)	11.4 s	116	~3.8 h
+ thread pipeline (v0.16.2)	11.1 s	110	~3.7 h
+ batched dts (v0.16.3, default)	10.9 s	107	~3.6 h
(pure model floor, micro-bench)	~9.3 s	94	~3.1 h

Backend choice

The defaults were picked by benching FILM-Style, FILM-L1, and RIFE 4.25 head-to-head on a real 2560×1440 broadcast frame from the test clips:

Backend	720p	1080p	1440p native	1440p tile-stitch	End-to-end @ 1440p
RIFE 4.25	16 ms	26 ms	40 ms	n/a	~250 ms/pair
FILM-Style	197 ms	411 ms	3919 ms (cliff)	825 ms	~1000 ms/pair
FILM-L1	199 ms	413 ms	14404 ms (cliff worse)	830 ms	~1000 ms/pair

RIFE is **~4x faster end-to-end** than FILM at 1440p and produces visually equivalent output on ultimate-frisbee broadcast footage (see `_spike_rife_vs_film_quality.py` for the side-by-side renders).

WARNING — selecting `--backend film`

FILM-Style produces slightly softer / more natural in-betweens on very large motion (a sprinter crossing several player-widths between frames). For 99% of cases this is not worth its ~4x runtime cost.

Approximate per-100-minute-match wall times on an RTX 2080 Ti:

Mode	RIFE	FILM
2x (40 fps)	~8 h	~33 h
3x (60 fps)	~16 h	~66 h
4x (80 fps)	~25 h	~133 h

`waruka interpolate --backend film` prints this warning at startup so it isn't picked by accident. Pick FILM only for special-occasion renders where you're prepared to wait days.

Source / target fps relationship

`target_fps` must divide cleanly into an integer multiple of source fps. From a 20-fps `waruka` render output:

	Output	In-betweens per source pair
40	40 fps	1 (dt=0.5)
60	60 fps (default)	2 (dt=1/3, 2/3)
80	80 fps	3 (dt=0.25, 0.5, 0.75)

Higher multipliers don't visibly help for camera-followed footage beyond 60 fps and quadruple/triple the compute. 60 fps is the recommended default for tactics review.

Where the models live

- `third_party/rife/` — Practical-RIFE clone + `train_log/flownet.pkl`

(RIFE 4.25, ~24 MB). Apache 2.0 / MIT.

- `third_party/film/film_net_fp16.pt` — FILM-Style TorchScript

(~66 MB, dajes PyTorch port of the Google Research model). Apache 2.0.

- `third_party/film/film_net_l1_fp16.pt` — FILM-L1 alternate

weights, currently not exposed via the CLI (no quality / speed advantage was found vs Style during the v0.14 bench).

The two backends are independent — each can be deleted to slim the distribution if only one is wanted.

Output progress

`waruka interpolate` writes the same `_progress.json` heartbeat as `waruka render`, so `waruka monitor` shows live progress (current pair, `fps_observed`, ETA).

[10] waruka upscale (v0.16)

Standalone 2x super-resolution on any input video using Real-ESRGAN x2plus. Mirrors `waruka interpolate`'s NVDEC -> GPU preprocess -> NVENC architecture; no pair-cache or batching -- SR is single-input per-frame and the model is the dominant cost by orders of magnitude.

```
python -m waruka upscale input.mp4 --out upscaled.mp4 [--weights PATH]
      [--fp32] [--t0 SEC] [--t1 SEC] [--no-nvdec] [--cq 23]
```

Arg	Default	Purpose
<code>input</code>	required	Source video (any codec OpenCV/NVDEC reads).
<code>--out</code>	required	Output mp4 path. Dimensions are exactly 2x the input; fps preserved.
<code>--weights</code>	(bundled)	Override Real-ESRGAN weights path. Default uses <code>third_party/realesrgan/weights/RealESRGAN_x2plus.pth</code> .
<code>--fp32</code>	off	Use float32 instead of float16. Slower; debugging.
<code>--t0, --t1</code>	None	Process only the time window [t0, t1] seconds.
<code>--no-nvdec</code>	(NVDEC on)	Force cv2 H264 decode. SR dominates wall time so the source-decode choice is mostly irrelevant.
<code>--cq</code>	23	NVENC constant-quality target (0-51, lower=better).

Performance: at full 1440p input, SR takes ~10-15 s/frame on an RTX 2080 Ti -- much slower than the in-renderer SR path (~1.2 s/frame on cropped regions) because we feed the whole frame at once. A 100-min match (120k frames) would be days; this command is intended for short clips (highlights, individual points, isolated calls). For long-form upscale, use a smaller input resolution or wait for hardware that can amortise the cost.

Output codec: H.264 caps at 4096 px in either dimension. A 2x upscale of any input >=2048 wide blows past that, so the writer switches to HEVC NVENC (8192 px max) automatically. For smaller inputs the H.264 path stays preferred for player compatibility.

GUI (v0.13+, recommended)

```
python -m waruka gui
```

PySide6 (Qt 6.11) tabbed shell. Four tabs as of v0.16:

- **Track** -- end-to-end single-clip processing

(calibrate -> markfield -> tracking params -> process).

- **Concat** -- join 5-min Reolink chunks into one match video

with optional trim.

- **Post-process** (v0.16) -- run `waruka interpolate` and/or

`waruka upscale` against any existing video. When both are ticked the source is upscaled first then interpolated (faster per-frame math than the reverse order).

- **Queue** (v0.16) -- overnight batch processor. Set up many

games end-to-end, then run sequentially while you sleep. Persistent across crashes / restarts; pause / resume at stage boundaries; retry from failed stage.

See GUI documentation for a screenshot walkthrough of each tab.

Track tab

End-to-end driver for the existing pipeline. On Open video... (or drag-drop a video onto the `Source video` box, v0.14):

1. **Calibrate** — opens the same OpenCV calibrate window as the CLI.

Auto-skipped if the existing `project.json` already has dewarp.

2. **Mark field** — opens the same OpenCV markfield window. Auto-skipped

if the project already has a homography.

3. **Track parameters** — exposes the most-tuned CLI flags as Qt widgets.

4. **Process** — runs `track` → `classify` → `campath` → `render` (default

sequential) or `waruka pipeline` (chunked, secondary opt-in).

Artefacts are written to `<source_dir>/waruka_tracking/<basename>/`; final tracked video is `<source_dir>/<basename>_tracked.mp4`. Live progress + Kill button via the same `_progress.json` mechanism the CLI `waruka monitor` uses.

Concat tab

Multi-clip concatenation + trim for matches recorded as 5-min Reolink chunks. Drag-drop file list with monospace columns (filename, recorded datetime parsed from the Reolink `_DST<date>_<time>` filename pattern, duration, audio indicator, codec, resolution, fps). Audio + codec consistency checks with mid-list remediation buttons. `ffmpeg -c copy` concat with live progress + Kill button. cv2-based scrubber for trim (, . < > for ±1s / ±10s; I / O for in/out markers; Space to play). Date-prefilled output name (YYYYMMDD placeholder). Optional `_no_audio.mp4` silent companion. On Save: auto-handover to the Track tab with the trimmed file pre-loaded.

Post-process tab (v0.16)

Run interpolation and/or upscaling against any existing video. Two checkboxes (Interpolate / Upscale 2x); each can be combined. When both are checked the source is upscaled first to a temp file then interpolated to the final output -- faster than the reverse order because SR runs once per source frame instead of three times per interp triplet. Stage progress shows in the log pane as Stage 1/2: Upscale 2x... / Stage 2/2: Interpolate... Temp intermediates are cleaned up on success, failure, or cancel.

Queue tab (v0.16)

Overnight batch processor. Set up many games end-to-end, then start the queue before sleeping. The Add-job dialog mirrors the Track tab:

- Drop input chunks (concatenated in order).
- Pick output broadcast path; project.json path auto-derives at

`<broadcast_dir>/waruka_tracking/<basename>/project.json`.

- Run Calibrate + Mark field inline (uses the first chunk as

source). Status pips flip to ✓ when done.

- "Reuse project from previous job" -- copies the previous job's

project path; intended for back-to-back games on the same mount.

- Open tracking parameters -- launches the same ParamsDialog the

Track tab uses, so stride / t0 / t1 / mode (sequential vs pipeline) / view-mode / SR / audio-companion all behave the same way as the existing flow.

- Optional interpolation (target fps / backend / CQ).

Each queued job runs as `concat? -> track -> classify -> campath -> render -> audio_mux? -> interpolate?`. Concat skips for single-input jobs; audio_mux inserts when the first chunk has audio (matches Track tab's behaviour). Mode = pipeline collapses track/classify/campath/render into a single waruka pipeline stage.

Persistence: queue lives at `~/waruka/queue.json`, atomic writes on every mutation. On reload, any job whose status was "running" flips to "interrupted" so the user can see + retry. Pause completes the current stage then stops at the next boundary; Resume kicks back off. Per-job log archived at `<artefact_dir>/job.log` (Open log... button).

All intermediates (JSON, concat list, concat'd video, silent render pre-mux, pipeline chunks, log) live in the artefact subdir; final outputs (`broadcast.mp4`, `broadcast_smooth.mp4`, `broadcast_no_audio.mp4`) stay at user-picked locations.